Cégep du Vieux Montréal

Plan-cadre de cours
420.B0 Techniques de l'informatique
420.B9 Techniques de l'informatique Intensif
Dispensé par : **Le département d'informatique**

Titre : **Programmation orientée objet I** No : **420-C34-VM et 420-C34-IN**	Préalables ou corequis : **Préalable pour 420-B0 : 420-C21-VM** **Préalable pour 420-B9 intensif : 420-C21-IN** Aménagement particulier : **Laboratoire de micro-ordinateurs**	Pondération : **3-2-2** Durée : **75 heures** Unités : **2⅓** Session : **Pour :** **420-C34-VM : 3e** **420-C34-IN : 1re**	Compétence(s) visée(s) : **00Q6 Exploiter les principes de la programmation orientée objet.** Atteinte de la compétence : ***Atteinte partielle (P) de la compétence. (75 heures / 150 heures)***

La place du cours dans le programme

Ce cours est obligatoire pour les étudiants du programme de Techniques de l'informatique et de Techniques de l'informatique (intensif). Il se donne à la troisième session pour le programme régulier et à la première session du programme intensif par des enseignants du département d'informatique.

Seule la compétence 00Q6 est visée dans ce cours et elle est partielle. L'autre cours contribuant à l'atteinte de cette compétence est : *420-C44-VM ou 420-C44-IN Programmation orientée objet II* à la session 4 pour le DEC régulier et à la deuxième session pour le DEC intensif. L'étudiant doit avoir réussi le cours préalable *420-C21-VM* ou *420-C21-IN Programmation II* pour suivre ce cours.

Ce cours vise l'apprentissage des concepts principaux de développement par objet. À l'intérieur de ce cours, avec de nombreux exercices, on apprendra à programmer des classes afin d'en produire des objets pouvant interagir entre eux dans le but de produire des projets novateurs.

Le développement de ces classes sera l'occasion d'apprendre et d'utiliser différentes structures de données ainsi que différentes collections d'objets selon le contexte. L'étudiant sera amené à jumeler son modèle objet ainsi créé avec des interfaces-utilisateurs rendant le projet plus dynamique. Finalement, l'étudiant sera mis en contact avec la notion de tests unitaires afin de valider le bon fonctionnement de ses projets.

Pour le programme Techniques de l'informatique, les intentions éducatives en formation spécifique sont les suivantes :

- Développer l'autonomie et favoriser la polyvalence;
- Développer le sens de l'abstraction;

Autres cours contribuant à l'atteinte de la compétence 00Q6 pour :

420.B0 Techniques de l'informatique

No. du cours	Titre	Pond.	Session	Statut de la compétence
420-C44-VM	Programmation orientée objet II	3-2-2	4e	P*

420.B9 Techniques de l'informatique Intensif

No. du cours	Titre	Pond.	Session	Statut de la compétence
420-C44-IN	Programmation orientée objet II	3-2-2	2e	P*

La démarche d'apprentissage favorisée dans ce cours :

Le cours se déroulera entièrement en laboratoire. La majorité des cours consisteront en une présentation des concepts orientés objet à l'étude agrémentée de nombreux exercices à réaliser à l'ordinateur visant ainsi une participation active des étudiants afin de favoriser leur apprentissage. Ces exercices de programmation permettront aussi aux étudiants de développer entre autres leur sens de l'abstraction.

À d'autres moments, certains cours et/ou parties de cours seront consacrés à la réalisation des travaux pratiques ou du projet de l'épreuve certificative de ce cours. À l'aide des informations fournies par l'enseignant, l'étudiant pourra les appliquer afin de réaliser une application logicielle complète contenant un modèle objet respectant les principes étudiés en classe, une/des interfaces graphiques correctement paramétrées et des tests unitaires s'assurant de la fonctionnalité de cette application.

Dans le cadre de ces activités, les étudiants devront apprendre à faire preuve d'autonomie dans la recherche de solutions.

• Promouvoir les valeurs d'équité en matière d'accès aux technologies informatiques. La compétence 00Q6 contribue à l'atteinte de différents buts généraux du programme : B1 Rendre l'élève efficace dans l'exercice d'une profession : • Lui permettre, dès l'entrée sur le marché du travail, de jouer les rôles, d'exercer les fonctions et d'exécuter les tâches et les activités associées à une profession, • Lui permettre d'évoluer adéquatement dans un milieu de travail (ce qui implique des connaissances et des habiletés techniques et technologiques en matière de communication, de résolution de problèmes, de prise de décisions, d'éthique, de santé et de sécurité, etc.); B3 Favoriser l'évolution de l'élève et l'approfondissement de savoirs professionnels. : • Lui permettre de développer son autonomie et sa capacité d'apprendre ainsi que d'acquérir des méthodes de travail, • Lui permettre de comprendre les principes sous-jacents aux techniques et aux technologies utilisées, • Lui permettre de développer sa faculté d'expression, sa créativité, son sens de l'initiative et son esprit d'entreprise, • Lui permettre d'adopter des attitudes essentielles à son succès professionnel, de développer son sens des responsabilités et de viser l'excellence; B4 Favoriser la mobilité professionnelle de l'élève : • Lui permettre d'adopter une attitude positive à l'égard des changements, • Lui permettre de se donner des moyens pour gérer sa carrière, notamment par la sensibilisation à l'entrepreneuriat.	Les moyens d'évaluation formative permettront à l'étudiant de s'exercer à atteindre l'objectif global du cours. Par des évaluations sommatives, il démontrera ses capacités à l'atteindre.

Objectif global : Ce cours vise à exploiter les principes de la programmation orientée. **Objectifs spécifiques :** À la fin du cours, l'étudiant sera en mesure de : • Analyser le problème. • Modéliser les classes. • Produire les algorithmes pour les méthodes. • Programmer des classes.	**Épreuve certificative** Pour les cours **420-C34-VM** et **420-C34-IN**, l'épreuve certificative se divise en deux volets qui permettront d'évaluer l'atteinte des éléments visés par la compétence de ces cours : 1. Un examen qui permettra de valider la capacité d'établir des liens avec la matière vue durant la session. L'examen combinera une partie écrite et une partie à réaliser à l'ordinateur consistant à programmer une application contenant des classes démontrant la capacité de l'étudiant à développer un modèle objet. et 2. Un projet où l'étudiant devra concevoir une application orientée objet en accord avec les notions vues en classe. Les bonnes pratiques de programmation générale, l'utilisation de principes orientés objet ainsi que

• Documenter la programmation.	le développement de tests unitaires adéquats seront des points importants de ce projet. Ce projet permettra à l'étudiant de démontrer : • Qu'il comprend le problème soumis. • Qu'il est capable de concevoir une application orientée objet. • Qu'il est capable de traduire l'algorithme dans un langage de programmation. • Qu'il est capable de tester et corriger son programme. • Qu'il est capable de produire une documentation complète. L'épreuve certificative de ces cours a une pondération totale de 50% de la note finale. De plus, ces cours sont à double seuil. **Contexte de réalisation de l'examen :** • Individuellement; • À la fin de la session; • En classe; • À l'aide des notes de cours. **Critères généraux d'évaluation de l'examen :** • Justesse des réponses; • Qualité du français. **Pondération de l'examen : 30** % **Contexte de réalisation du projet :** • Individuellement; • Vers la fin de la session; • En classe et en dehors des heures de classe; • À partir d'une problématique dictée par l'enseignant; • À l'aide de logiciels appropriés; • À l'aide de la documentation pertinente; • Plusieurs étapes avec possibilité de remise à la fin de chacune des étapes. **Critères généraux d'évaluation du projet :** • Qualité générale du code (optimisation); • Utilisation de concepts orientés objet (héritage, polymorphisme); • Fonctionnalités demandées présentes; • Commentaires pertinents; • Développement de tests unitaires; • Qualité et pertinence de la documentation; • Qualité du français. **Pondération du projet : 20** %

Les éléments de compétences, les objectifs d'apprentissages ainsi que les balises de contenus traités dans le cours sont indiqués en caractère gras.

Code de la compétence : **00Q6**	Énoncé de la compétence : **Exploiter les principes de la programmation orientée objet.**	Objectifs d'apprentissage liés à l'ensemble de la compétence : **Aucun**
Contexte de réalisation / Précisions : • **À partir d'un problème.** • **À l'aide de règles de nomenclature et de codage.**		
Éléments de la compétence	**Objectifs d'apprentissage**	**Balises de contenu**
1. Analyser le problème.	**1.1 Décomposer le problème en fonction des exigences de l'approche orientée objet.**	**1.1.1 Définition de l'approche orientée objet.** **1.1.2 Présentation et reconnaissance des stratégies permettant de résoudre un problème à l'aide de l'approche orientée objet.** **1.1.3 Méthodologie d'analyse orientée objet : apprendre à reconnaître / identifier les objets intervenants dans l'espace problème.** **1.1.4 Méthodologie de traduction de la représentation fonctionnelle du problème selon le paradigme orienté objet.** *1.1.5 Explication de l'utilisation d'un regard critique à propos de l'analyse.*
	1.2 Déterminer correctement des données d'entrée, des données de sortie et de la nature des traitements.	**1.2.1 Présentation et énumération des caractéristiques des différents types de données.** **1.2.2 Distinction des différents types de données.** **1.2.3 Détermination du lien entre les données et les objets identifiés.** *1.2.4 Explication du modèle pour faire l'analyse des données d'entrée et de sortie.* *1.2.5 Détermination des comportements agissant sur les données.* *1.2.6 Méthodologie d'analyse de la nature des traitements selon les données d'entrée et des résultats attendus en données de sortie.* *1.2.7 Vérification de l'intégrité des données.*
	1.3 Déterminer de façon juste des classes à modéliser.	**1.3.1 Définition des termes spécifiques.** **1.3.2 Méthodologie de construction de schémas initiaux des classes à partir de l'identification préalable des objets identifiés lors de l'analyse.** **1.3.3 Identification des besoins en termes d'attributs et de méthodes.**

Code de la compétence : **00Q6**	Énoncé de la compétence : **Exploiter les principes de la programmation orientée objet.**	Objectifs d'apprentissage liés à l'ensemble de la compétence : **Aucun**
Contexte de réalisation / Précisions : • **À partir d'un problème.** • **À l'aide de règles de nomenclature et de codage.**		
Éléments de la compétence	**Objectifs d'apprentissage**	**Balises de contenu**
		1.3.4 Présentation et explication des notions d'encapsulation, d'agrégation, de composition, d'association, d'héritage, de polymorphisme et de classes abstraites. **1.3.5 Méthodologie d'application des modificateurs d'accès d'une variable ou d'une méthode.** **1.3.6 Méthodologie d'utilisation des membres statiques de classe.**
	1.4 Déterminer correctement des algorithmes à produire.	**1.4.1 Modélisation de la solution selon l'analyse du problème.** **1.4.2 Énumération et caractérisation de l'ensemble des méthodes.** **1.4.3 Présentation des critères de performance des algorithmes pour les reconnaître et pouvoir les appliquer.** **1.4.4 Spécification des fonctions à adapter dans les algorithmes en fonction de la complexité et de la taille du problème ou de la tâche.** **1.4.5 Méthodologie d'association correcte de l'algorithme à produire aux objets qui doivent réaliser la tâche.** **1.4.6 Démonstration et pratique du déroulement des algorithmes avec et sans ordinateur.**
2. Modéliser les classes.	**2.1 Déterminer correctement des attributs et des méthodes des classes.**	**2.1.1 Définition de termes spécifiques.** **2.1.2 Définition et caractérisation des types prédéfinis versus des classes.** **2.1.3 Méthodologie d'application de références à des objets comme attributs.** **2.1.4 Présentation et méthodologie sur les notions de surcharge de méthodes.**
	2.2 Appliquer judicieusement des principes d'encapsulation et d'héritage.	**2.2.1 Explication des principes d'encapsulation et d'héritage.** **2.2.2 Explication de la méthodologie de réutilisation de codes et des méthodes par héritage.** **2.2.3 Explication de la méthodologie de codage de classes et des méthodes abstraites.** **2.2.4 Distinction de l'héritage simple et de l'héritage multiple.**
	2.3 Représenter graphiquement et correctement des classes et de leurs relations.	**2.3.1 Apprentissage et développement des diagrammes de classes avec un système de représentation symbolique.**

Code de la compétence : **00Q6**	Énoncé de la compétence : **Exploiter les principes de la programmation orientée objet.**	Objectifs d'apprentissage liés à l'ensemble de la compétence : **Aucun**
Contexte de réalisation / Précisions : • **À partir d'un problème.** • **À l'aide de règles de nomenclature et de codage.**		
Éléments de la compétence	**Objectifs d'apprentissage**	**Balises de contenu**
	2.4 Respecter des règles de nomenclature.	**2.4.1 Présentation et caractérisation des librairies de base du langage.** *2.4.2 Méthodologie d'analyse des usages pour sélection lorsque diverses règles peuvent s'appliquer.*
3. Produire les algorithmes pour les méthodes.	**3.1 Déterminer adéquatement des opérations nécessaires pour chaque méthode.**	**3.1.1 Énumération des méthodes pour produire des algorithmes.** **3.1.2 Énumération des opérations nécessaires pour produire des algorithmes pour chacune des méthodes.** **3.1.3 Présentation et caractérisation des contextes dont l'objet ou la classe recevront des messages.** **3.1.4 Explication sur les attributs qui seront manipulés et comment ils interagiront avec les paramètres.**
	3.2 Déterminer correctement une séquence logique des opérations.	**3.2.1 Méthodologie de séquençage correcte des énoncés permettant de résoudre une problématique entre autres grâce aux méthodes d'autres objets.** **3.2.2 Méthodologie de regroupement des instructions qui expriment un traitement précis en une méthode.** **3.2.3 Explication des paramétrages des méthodes de manière à rendre leurs usages plus génériques.**
	3.3 Vérifier de façon appropriée le fonctionnement des algorithmes.	**3.3.1 Méthodologie de représentation du déroulement de l'algorithme sans exécuter le code.** **3.3.2 Validation du fonctionnement de l'algorithme.** **3.3.3 Méthodologie de vérification du fonctionnement des algorithmes.** **3.3.4 Méthodologie d'insertion de tests.** **3.3.5 Méthodologie de gestion des plans de tests.**
	3.4 Représenter correctement des algorithmes.	**3.4.1 Méthodologie d'implantation de l'algorithme dans un langage orienté objet.**
4. Générer l'interface graphique.	*4.1 Choisir de façon appropriée des éléments graphiques pour l'affichage et la saisie.*	*4.1.1 Énumération des étapes de planification de l'aspect graphique des interfaces.* *4.1.2 Caractérisation de la technique de dessin sur mesure et l'utilisation de la technique avec les composantes graphiques préétablies.* *4.1.3 Présentation des différents types de composants.*

Code de la compétence : **00Q6**	Énoncé de la compétence : **Exploiter les principes de la programmation orientée objet.**	Objectifs d'apprentissage liés à l'ensemble de la compétence : **Aucun**
Contexte de réalisation / Précisions : • **À partir d'un problème.** • **À l'aide de règles de nomenclature et de codage.**		
Éléments de la compétence	**Objectifs d'apprentissage**	**Balises de contenu**
		4.1.4 Définition de l'anticrénelage. *4.1.5 Méthodologie d'utilisation de l'anticrénelage.*
	4.2 Positionner correctement des éléments graphiques.	*4.2.1 Présentation et discussion d'aspects ergonomiques.* *4.2.2 Méthodologie de positionnement des éléments selon leur taille et le système de coordonnées.* *4.2.3 Présentation et caractérisation de différents « layouts ».* *4.2.4 Méthodologie d'adaptation de l'aspect et du positionnement des composantes en cours d'exécution.*
	4.3 Paramétrer correctement des éléments graphiques.	*4.3.1 Méthodologie de modification des couleurs et des polices.* *4.3.2 Explication de l'importance de prendre en compte de l'orientation de l'appareil.* *4.3.3 Explication de l'importance de prendre en compte de la langue utilisée sur l'appareil.* *4.3.4 Présentation des contraintes possibles par rapport aux composantes.* *4.3.5 Méthodologie d'établissement des contraintes par rapport aux composantes parentes ou voisines.*
5. Programmer des classes.	**5.1 Choisir de façon appropriée des instructions, des types de données élémentaires et des structures de données.**	**5.1.1 Méthodologie de déclaration des variables d'instance.** **5.1.2 Méthodologie de déclaration des constantes.** **5.1.3 Définition et caractérisation de constructeur.** **5.1.4 Méthodologie pour coder des constructeurs.** *5.1.5 Caractérisation dans le but de faire la distinction entre différents types de structures de données.* *5.1.6 Méthodologie de gestion des exceptions.* *5.1.7 Méthodologie de création des classes d'exception.* *5.1.8 Utilisation des patrons / des génériques.* *5.1.9 Méthodologie pour déterminer quelles sont les données qui devront être conservées.* *5.1.10 Explication et méthodologie de création des algorithmes itératifs dans le paradigme orienté objet.* *5.1.11 Explication et méthodologie de création des algorithmes récursifs dans le paradigme orienté objet.* *5.1.12 Différenciation de l'approche itérative avec l'approche récursive.*

Code de la compétence : **00Q6**	Énoncé de la compétence : **Exploiter les principes de la programmation orientée objet.**	Objectifs d'apprentissage liés à l'ensemble de la compétence : **Aucun**
Contexte de réalisation / Précisions : • **À partir d'un problème.** • **À l'aide de règles de nomenclature et de codage.**		
Éléments de la compétence	**Objectifs d'apprentissage**	**Balises de contenu**
		5.1.13 Explication de l'approche diviser pour mieux régner. *5.1.14 Familiarisation avec les rudiments de la complexité.*
	5.2 Organiser logiquement des instructions.	**5.2.1 Méthodologie d'implantation de solutions orientées objet itératives et récursives à des problèmes concrets.** *5.2.2 Méthodologie de gestion des données à l'aide de structures de données.* *5.2.3 Méthodologie de création des algorithmes pour manipuler les structures de données en pseudo-code et ensuite dans un langage de programmation orienté objet.*
	5.3 Programmer correctement des messages à afficher à l'utilisatrice ou à l'utilisateur.	*5.3.1 Méthodologie de création de boîtes de dialogues.* *5.3.2 Explication pour positionner la boîte de dialogue par rapport à la fenêtre principale.* *5.3.3 Méthodologie de gestion des événements survenus sur la boîte de dialogue.* *5.3.4 Méthodologie de transfert des informations de la boîte de dialogue au programme principal si nécessaire.*
	5.4 Intégrer correctement des classes dans le programme.	*5.4.1 Connaissance et méthodologie d'application de différentes architectures connues.* *5.4.2 Connaissance des différents objets événementiels et des informations qu'ils renferment.* *5.4.3 Connaissance des différentes interfaces-écouteur disponibles et de leurs utilités.* *5.4.4 Connaissance et méthodologie d'utilisation de patrons de conception simples.* *5.4.5 Détermination du rôle de la classe et de la méthodologie à restreindre la classe à ce rôle.* *5.4.6 Explication d'exposition uniquement des attributs et des méthodes nécessaires.* *5.4.7 Méthodologie d'utilisation de l'héritage afin de contraindre le comportement de certaines classes et d'assurer une régularité des messages.* *5.4.8 Utilisation de l'héritage pour éviter la redondance.* *5.4.9 Définition des sous-classes de classes de composantes.*

Code de la compétence : **00Q6**	Énoncé de la compétence : **Exploiter les principes de la programmation orientée objet.**	Objectifs d'apprentissage liés à l'ensemble de la compétence : **Aucun**
Contexte de réalisation / Précisions : • **À partir d'un problème.** • **À l'aide de règles de nomenclature et de codage.**		
Éléments de la compétence	**Objectifs d'apprentissage**	**Balises de contenu**
		5.4.10 Méthodologie de gestion des événements à l'aide d'écouteurs et des informations contenues dans les objets événementiels.
	5.5 Voir au fonctionnement correct du programme.	*5.5.1 Explication des erreurs de compilation qui surviennent et explication pour les éviter.* *5.5.2 Méthodologie d'utilisation d'un débogueur dans un contexte orienté objet.* *5.5.3 Explication de l'importance des tests.* **5.5.4 Méthodologie de développement des tests unitaires.** **5.5.5 Méthodologie d'élaboration des suites de tests.** *5.5.6 Connaissance des erreurs survenant lors de l'exécution (exceptions).* *5.5.7 Connaissance sur la gestion des situations exceptionnelles.* *5.5.8 Connaissance de la structure de gestion des erreurs et des exceptions (try / catch).* *5.5.9 Méthodologie de vérification de l'efficacité de la fermeture de l'application.* *5.5.10 Méthodologie d'utilisation et de configuration des émulateurs.*
	5.6 Respecter la syntaxe du langage.	**5.6.1 Connaître les conventions du langage pour les respecter.** **5.6.2 Familiarisation avec les particularités du langage.** **5.6.3 Explication de l'effet des erreurs de syntaxe.** **5.6.4 Méthodologie de structuration des instructions à l'intérieur du cadre du langage.** **5.6.5 Connaissance des mots clés du langage.**
	5.7 Respecter des règles de codage.	**5.7.1 Définition et méthodologie pour indenter.** **5.7.2 Méthodologie d'encapsulage du code à l'aide de classes et de méthodes.** **5.7.3 Explication pour éviter la redondance.** **5.7.4 Méthodologie pour nommer significativement aux entités informatiques d'un programme.** **5.7.5 Connaissance des conventions de programmation.**
6. Documenter la programmation.	**6.1 Faire les notations clairement de commentaires dans le code informatique.**	**6.1.1 Énumération et présentation des différentes informations significatives à rédiger.**

Code de la compétence : **00Q6**	Énoncé de la compétence : **Exploiter les principes de la programmation orientée objet.**	Objectifs d'apprentissage liés à l'ensemble de la compétence : **Aucun**
Contexte de réalisation / Précisions : • **À partir d'un problème.** • **À l'aide de règles de nomenclature et de codage.**		
Éléments de la compétence	**Objectifs d'apprentissage**	**Balises de contenu**
		6.1.2 Méthodologie de rédaction des aspects à noter dans le code informatique. *6.1.3 Connaissance des annotations propres au langage.*
	6.2 Faire les notations clairement de la documentation d'aide à la programmation.	**6.2.1 Présentation des notions à spécifier dans la documentation d'aide : déclarer l'auteur, la version, le type de retour des méthodes.** *6.2.2 Utilisation d'un logiciel de rédaction de notes du codage.* *6.2.3 Démonstration du codage de liens hypertexte dans les fichiers d'aide.*
	6.3 Utiliser de façon appropriée des générateurs de documentation.	*6.3.1 Présentation d'un générateur de documentation.* *6.3.2 Méthodologie d'utilisation d'un générateur de documentation.* *6.3.3 Présentation et caractérisation des paramètres.*
7. Appliquer la procédure liée à la gestion des versions de programmes.	*7.1 Configurer correctement le système de gestion de versions.*	*7.1.1 Définition de termes spécifiques.* *7.1.2 Méthodologie de création de dépôts.* *7.1.3 Méthodologie de gestion des branches.* *7.1.4 Méthodologie de gestion du partage.* *7.1.5 Méthodologie de gestion des fichiers à exclure.* *7.1.6 Méthodologie d'identification des fichiers selon un système de gestion.*
	7.2 Faire une soumission méthodique du code modifié.	*7.2.1 Définition de termes spécifiques.* *7.2.2 Démonstration de l'importance de soumettre des modifications atomiques.* *7.2.3 Méthodologie de résolution des bogues en retournant à des versions fonctionnelles du programme.* *7.2.4 Explication de l'effet de conflits.*
	7.3 Gérer judicieusement des branches et des conflits.	*7.3.1 Situations où développer des branches sont appropriées.* *7.3.2 Fusion de la branche avec la branche par défaut.* *7.3.3 Lecture et compréhension des marques de résolution de conflit.*